COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

Ownership, abstractions and C++ design

Module 12 / expanded Volume 03

Four lessons, worked examples, full practical requirements and 20 lesson questions.

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

12.1 — RAII and resource ownership 3

12.2 — Copy, move and exception safety 7

12.3 — Interfaces, composition and polymorphism 10

12.4 — Templates, concepts and data-oriented choices 13

Continuity with the original program 16

Primary reference directory 16

Reading by topic 18

Module 12 — Ownership, abstractions and C++ design

Prerequisites: Module 11.

Project: Build a small resource manager with explicit ownership, safe move behavior, bounded interfaces, fault-injection tests and a written comparison of composition and inheritance.

Primary references: R11, R12. Four lessons,20 expanded quiz questions and a module project that contributes to GateC.

12.1 — RAII and resource ownership

Learning outcome: Make cleanup follow object lifetime on success and failure paths.

A resource is more than heap memory

Files, device handles, locks and temporary registrations all have acquisition and release rules. Resource Acquisition Is Initialization connects successful acquisition to an object's lifetime. A constructed owner is responsible for release when it leaves scope, including ordinary early returns and exception unwinding. The destructor provides the cleanup path rather than requiring every caller to remember it.

The resource-owning object must know whether it actually acquired the resource. A default-constructed or moved-from owner may be empty. Its destructor must not close an invalid handle. If construction fails after an owning member is fully constructed, that member is destroyed, even though the enclosing object's destructor is not run for the incomplete object.

Prefer established owning members such as `std::unique_ptr` and standard containers. The custom mock owner in this lesson is educational: it exposes acquisition/release counts so you can verify the invariant. Real graphics-device wrappers add device affinity, synchronization and destruction-order constraints beyond this mock. [R11: R, E]

One owner is usually easier to reason about

Unique ownership means one object controls the resource's release. A borrowed pointer can refer to the resource temporarily without participating in ownership. Shared ownership has a different contract: the resource survives until the last owning reference is released. That can be useful, but reference counting and delayed destruction introduce costs and timing obligations.

Owning cycles can prevent shared objects from reaching a reference count of zero. Break a relationship that is semantically observational with a weak or non-owning link; do not make every edge owning by default. More fundamentally, ask which subsystem should govern the resource's lifetime.

A resource manager often owns assets while users hold stable IDs or scoped borrows. The choice depends on whether an asset may be unloaded while a job still uses it. This volume stays single-threaded, but the ownership diagram should already expose where concurrency would need additional coordination.

Failure paths are first-class behavior

Imagine opening a file, then allocating a buffer, then parsing metadata. Parsing can fail after both acquisitions. Manual code needs a correct release on every exit, including future exits added by maintenance. RAII owners make the cleanup order follow scope instead.

The lab tracks acquired, released and currently live resources. After a normal operation, live count returns to the baseline. After an injected failure immediately after acquisition, it also returns to baseline. The important assertion is not merely that the program did not crash; it is that every successful acquisition was released exactly once.

Cleanup must not throw from an ordinary destructor used during stack unwinding. If a release operation can report an important failure, provide an explicit close/flush operation that returns or throws in a controlled context, and make destructor cleanup a documented fallback. A mock counter cannot establish the behavior of a real device API.

Respect destruction dependencies

C++ members are initialized in their declaration order and destroyed in reverse declaration order, regardless of the order written in the constructor's initializer list. If an asset owner needs the device alive during release, declare and construct the device before the dependent asset owners, or use a higher-level scope that enforces the same dependency.

A borrowed view must not outlive its resource owner. RAII solves release ownership, not all dangling borrows. An object's destructor can be perfectly correct while another subsystem still keeps a pointer to the destroyed resource. That is why the earlier ownership diagram remains necessary.

In a game shutdown sequence, stop new work, finish or cancel outstanding uses, release dependent resources, and finally release the underlying device. Later systems modules will add job synchronization and GPU completion. Here you must articulate the order without pretending the mock device has those real-world guarantees.

Inject a failure after acquisition

Begin with live = 0, acquired = 0 and released = 0. Construct the mock owner: live = 1, acquired = 1. Trigger a controlled exception before normal completion. Stack unwinding destroys the owner: live = 0, released = 1. The outer catch records the failure. Repeat with a successful path and verify the counts advance to acquired = released = 2.

Now create an empty owner. Its destruction must leave the counters unchanged. These three paths distinguish successful acquisition, failed work after acquisition and no acquisition at all.

For an enclosing object whose second member fails to construct, trace which earlier member owners already exist. Their destructors still run; the incomplete outer object does not get its normal destructor call.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_12_1.cpp`. Pseudocode excerpts are not standalone C++ source.

```
void load_with_failure(bool fail_after_acquire) {
    MockOwner resource;
    if (fail_after_acquire)
        throw std::runtime_error("injected load failure");
    // resource is released at scope exit on either path
}
```

Common failure modes

RAII does not make a borrowed pointer immortal. Shared ownership is not a universal leak cure. Constructor initializer-list order does not override member declaration order. A destructor that throws during unwinding can terminate the process.

Practical assignment

Original requirement: Wrap a mock file/device resource whose open/close counters can be inspected. Inject failure after acquisition. Pass: each acquired resource is released exactly once and no view outlives its owner.

1. Implement or inspect a mock resource with observable acquisition and release counters.
2. Wrap it in a non-copying RAII owner and test normal completion, early return and injected failure.
3. Add an enclosing object with dependent members and explain partial-construction cleanup.
4. Draw the device-before-resource construction and resource-before-device destruction order.

Required cases: successful scope releases exactly once; injected exception releases exactly once; empty owner releases nothing; nested owners unwind in reverse order; all paths return live count to baseline.

Evidence to submit: Acquisition/release ledger, fault-injection tests, ownership diagram and destruction-order explanation.

Independent transfer: A file close operation can fail when flushing buffered data. Separate explicit error-reporting close from destructor cleanup without allowing two releases.

Assigned primary reading: R11: Resource Management and smart-pointer ownership rules.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What does RAII connect?
2. Can shared ownership create a leak through cycles?
3. Why must device-resource destruction order be planned?

4. Does an enclosing object whose constructor throws get its normal destructor call?
5. Why can correct RAII still coexist with a use-after-free bug?

12.2 — Copy, move and exception safety

Learning outcome: Define what transferring a resource means without duplicating ownership.

Copying a value is not duplicating an owner

A value such as a position can be copied naturally. A device handle often represents a unique obligation to release one resource. Copying its integer handle into another independent owner would give two destructors the same obligation. Delete copy operations for a unique owner unless the resource API supplies a real duplication operation with clear semantics.

A move transfers ownership into a destination and leaves the source in a valid state. For the educational handle, the source becomes empty. Standard-library moved-from objects commonly have a valid but unspecified state unless their contract says more; do not generalize the mock's empty-state guarantee to every type.

`std::move` is a cast that allows move-aware overload selection. It does not itself transfer bytes, release anything or guarantee that a move overload exists. Moving a const object can select a copy because ordinary moves need to modify the source. Read the selected operation rather than treating the word move as a magical effect. [R11: C.20, C.21, C.64]

Prefer the rule of zero; learn special members deliberately

The rule of zero means composing a class from members that already implement the desired ownership semantics so custom copy, move and destruction are unnecessary. A vector of values and a unique pointer to a mock resource can manage themselves. Writing five special member functions for every class increases the number of invariants you must maintain.

When a custom handle wrapper is justified, make its contract explicit: default construction is empty; copying is prohibited; move construction steals ownership; move assignment releases the old destination resource before taking the source; destruction releases a nonempty handle. Guard self-move assignment or define an implementation that remains valid under it.

Mark a move `noexcept` only when it really cannot throw. Containers can use this information when deciding how to preserve their own guarantees during relocation. A false `noexcept` promise is dangerous: throwing through it terminates instead of allowing ordinary recovery.

Three useful levels of failure guarantees

The basic exception guarantee preserves valid invariants and avoids leaks, but values may change. The strong guarantee leaves relevant observable state unchanged on failure. A no-throw guarantee promises the operation does not propagate an exception. These are interface commitments, not optimistic comments.

For asset replacement, load and validate a candidate without touching the current asset. Only after all fallible preparation succeeds should a nonthrowing commit swap the candidate into the current slot. If loading fails, the old asset remains accessible. After successful commit, the old asset is released through the temporary candidate's destruction.

Define what observable state means. Logging a failed attempt or incrementing a diagnostic counter may be permitted even while the asset binding remains unchanged. State the boundary so tests do not confuse the asset guarantee with a promise that absolutely no bit anywhere changes.

Failure injection tests the design, not luck

A successful run tests only the successful path. Add controlled failures after acquisition, during validation and before commit. For each point, assert the old asset ID and payload remain unchanged, live-resource count returns to its expected baseline, and no resource is released twice.

Do not simulate allocation failure by exhausting the host machine. Use a deterministic injectable mock loader. The tests should fail at a named step and produce a reproducible state transition. Later you can test a real allocator or device boundary in an isolated integration test.

A strong guarantee can cost extra memory because old and new resources coexist before commit. That is a tradeoff to quantify, not a reason to abandon failure correctness silently. A memory-constrained engine might publish a weaker guarantee with an explicit fallback state.

Transactional replacement of asset A by asset B

The current owner holds A. Construct candidate B independently. If B's validation throws, candidate cleanup releases B and current still owns A. If validation succeeds, swap the two owners. Current now owns B; the candidate temporarily owns A and releases it at scope exit.

At peak, both assets are live. After completion, only the selected current asset remains live. Moving an owner into a second variable transfers its release obligation but does not load a second underlying asset.

Test move assignment into a destination that already owns C: C must be released exactly once before the destination takes the new resource. Self-move must not duplicate ownership or corrupt the wrapper's invariant.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_12_2.cpp`. Pseudocode excerpts are not standalone C++ source.

```
void replace_asset(AssetOwner& current, Loader& loader) {
    AssetOwner candidate = loader.load_and_validate();
    current.swap(candidate); // documented nonthrowing commit
}
```

Common failure modes

A moved-from object is not universally empty. Declaring a destructor can affect implicitly generated move operations. `noexcept` is a promise, not a request for the compiler to make code safe. Testing only move construction misses move-assignment leaks.

Practical assignment

Original requirement: Implement a move-only mock resource and transactional asset replacement. Pass: copying is prohibited, moving does not double-release, and injected load failure preserves the old asset.

1. Implement a move-only mock owner or explain a rule-of-zero alternative.
2. Add compile-time tests proving copying is prohibited and move construction is supported.
3. Build a transactional replacement function and inject failures before commit.
4. Measure the maximum simultaneous live-resource count and state the memory tradeoff.

Required cases: copy constructible is false; move leaves exactly one owning handle; move assignment releases prior destination; self-move remains valid; load/validation failure preserves old asset; successful replacement releases old asset once.

Evidence to submit: Ownership transfer trace, compile-time constraints, failure matrix and a statement of basic/strong/no-throw guarantees.

Independent transfer: A reload operation updates both an asset and an ID index. Show why swapping only the asset may not provide the strong guarantee for the whole repository.

Assigned primary reading: R11: class special members, rule of zero and exception safety.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does std::move itself guarantee that an object is moved?
2. What is the strong exception guarantee?
3. Why prefer the rule of zero?
4. Why might transactional replacement need more peak memory than destructive replacement?
5. What happens when an exception escapes a function declared `noexcept`?

12.3 — Interfaces, composition and polymorphism

Learning outcome: Choose abstractions according to substitutability and measured cost.

An interface is a behavioral promise

Two implementations are interchangeable only when they satisfy the same observable contract. A renderer interface might accept a bounded view of visible units and emit a frame without changing simulation state. Whether the backend prints text or records commands is secondary; input validity, ordering and error behavior must match.

Substitutability is stronger than matching method names. A recording renderer that silently sorts units changes observable order if the contract promises input order. A backend that stores a borrowed span after the call changes the lifetime obligation. Specify whether the method consumes the data synchronously or retains owned copies.

Keep interfaces narrow enough to test. A renderer should not need to own the entire game world just to draw a few unit positions. This lesson uses a non-owning renderer reference for one immediate call and a separately owned backend. The ownership arrangement remains visible. [R11: I, C]

Composition and runtime polymorphism solve different problems

Composition assembles behavior from members or collaborators: a simulation uses a renderer and a repository. Inheritance expresses an interface relationship or carefully designed implementation relationship. Prefer a clear has-a relationship over an inheritance hierarchy built merely to share fields.

Runtime polymorphism through a virtual interface supports substituting a backend whose concrete type is selected at runtime. A base intended for deletion through a base pointer needs suitable virtual destruction. An alternative is a protected nonvirtual destructor when deletion through the base is forbidden; the design must make that policy enforceable.

Do not claim virtual calls are always too slow or always free. Their cost depends on dispatch, inlining opportunities, data access and call frequency. A single virtual call submitting a batch differs from one virtual call per component in an inner loop. The next lesson introduces static generic interfaces and data-layout experiments.

A recording implementation makes tests observable

A recording renderer copies each submitted command into an owned vector. Tests can compare the resulting command sequence without opening a window. A console renderer writes the same logical commands to an output stream supplied by its caller. Both are checked against the same expected IDs and positions.

The recording implementation is not a fake that can ignore the contract. It should preserve ordering, reject the same invalid inputs when validation belongs to this boundary and document any intentional difference such as output destination. Tests that pass only because a mock behaves more permissively can hide integration defects.

Inject an output stream rather than writing directly to a global console object. That permits an in-memory stream in a test and clarifies which subsystem owns the output destination. Avoid a global service locator for this small example; explicit parameters make dependencies reviewable.

Separate architecture decisions from fashion

Write a brief decision record: the required variability, candidate designs, ownership model, error behavior and measurements that could overturn the choice. Candidates might include a virtual interface, a template parameter, a tagged variant or a direct concrete class. No single mechanism wins all requirements.

For a classroom backend selected at runtime, a small virtual interface is reasonable. For a fixed inner-loop mathematical kernel, a constrained template may expose optimization opportunities. For a closed set of alternatives, a variant can make exhaustive handling visible. Choose from the constraints rather than using every abstraction feature in one example.

The resource-manager project asks for a written comparison of composition and inheritance. Ground that comparison in your actual code: which state changes together, which dependencies have independent lifetimes, and which tests become simpler when responsibilities are separated?

One simulation, two renderers

Create units in the order [7,2]. The contract says `render` submits commands in that order and does not retain borrowed unit storage. A console backend writes `7:2,3` then `2:4,5`; a recording backend stores two equivalent command values. Modifying the source unit vector after the call cannot change already recorded commands.

Own a backend through `std::unique_ptr<Renderer>`, call through `Renderer&`, and let destruction occur through the base owner. The base's virtual destructor ensures the derived cleanup runs. The simulation itself neither allocates nor deletes the borrowed renderer reference.

A test checks ordering and copied values, not just the number of calls. That is what makes substitutability observable.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_12_3.cpp`. Pseudocode excerpts are not standalone C++ source.

```
struct Renderer {
    virtual ~Renderer() = default;
    virtual void draw(std::span<const Unit> units) = 0;
};
void frame(Renderer& renderer, std::span<const Unit> units) {
    renderer.draw(units); // borrowed for this call only
}
```

Common failure modes

An interface without lifetime rules is incomplete. A mock that ignores errors is not equivalent behavior. A base destructor need not always be virtual, but deletion through a base must be deliberately safe or deliberately forbidden.

Practical assignment

Original requirement: Create interchangeable console and recording renderers behind one small interface. Pass: both satisfy the same contract, tests run without display output, and ownership/ destructor behavior is explicit.

1. Write a one-page renderer contract before either implementation.
2. Implement console and recording backends with the same unit ordering and synchronous-borrow policy.
3. Test both through the interface with empty and multiple-unit inputs.
4. Write an architecture decision comparing runtime polymorphism with composition and a static alternative.

Required cases: empty input emits no commands; two units preserve input order; recorded commands own copied data; derived cleanup observed through base ownership; simulation does not mutate renderer ownership.

Evidence to submit: Interface contract, two backend implementations, shared contract tests and design decision record.

Independent transfer: A backend queues rendering asynchronously. Amend the contract to transfer owned command data or guarantee a valid lifetime until completion.

Assigned primary reading: R11: Interfaces, Class Hierarchies and composition guidance.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What does substitutability require?
2. Why can a polymorphic base need a virtual destructor?
3. Why not put every responsibility into one engine class?
4. A backend retains a span although the interface promises use only during the call. Is matching output today sufficient?
5. Why might one virtual call per batch have a different cost from one per entity?

12.4 — Templates, concepts and data-oriented choices

Learning outcome: Use generic code without hiding costs or exploding complexity.

Generic code still needs a semantic contract

A function template describes a family of functions. A C++20 concept constrains which type expressions are accepted by an interface and can improve diagnostics. It does not automatically prove every runtime property. A type meeting an ordered concept can still supply problematic values such as NaN in floating-point use.

For a teaching clamp, require an arithmetic domain you can explain, validate that the upper bound is not below the lower bound, and choose a nonfinite-value policy. The included example constrains an integer clamp to integral types and tests invalid bounds explicitly. This avoids silently presenting a generic floating-point contract it has not specified.

A template definition normally must be visible where it is instantiated, which is why many small generic functions live in headers. That can increase compilation work and expose more implementation detail. Explicit instantiation and separate implementation strategies exist, but are beyond this introductory example. [R11: T; P4-01]

Static and dynamic choices have different costs

A template-selected operation can allow specialization and inlining when the compiler sees the concrete types. It can also generate multiple versions, increase binary size and make diagnostics difficult. Runtime dispatch can centralize code and choose behavior at runtime, but may limit optimization at some call sites.

Do not equate the absence of virtual functions with good design. A deeply nested generic engine that takes minutes to compile and is hard to debug has costs outside its frame time. Record runtime, build time, binary size and maintenance constraints when those are relevant to your decision.

The goal is to express a simple contract without hiding its costs. A short constrained function is a better starting point than a universal entity framework. The module project should remain small enough that the learner can explain every ownership path.

Array of structures and structure of arrays

An array of structures stores complete entities next to one another: position, velocity and other fields for entity 0, then entity 1. A structure of arrays groups the same field across entities. If a hot loop reads only x position and x velocity, grouping those fields can reduce unrelated data traffic. If work uses every field of one entity together, the tradeoff can differ.

Both layouts can preserve exactly the same simulation semantics. Keep entity count, arithmetic, update count and output validation equivalent. A benchmark that updates fewer fields in one version is not comparing layout alone. Allocations, initialization and result validation should be placed consistently outside or inside the timed region according to the declared experiment.

The provided small benchmark emits CSV and a checksum. Its output is a local measurement, not a universal ranking. Real hardware adds cache levels, prefetching, vectorization and bandwidth effects studied in later volumes. [R15; R11: Per]

Design a measurement that can be contradicted

Write a hypothesis before timing: for this x-only update over this number of entities, layout A might move less unrelated data than layout B. Record compiler, optimization flags, CPU identity, input count and repetitions. Run multiple samples, alternate order when practical and inspect the distribution rather than reporting a single best number.

Use a monotonic clock for elapsed intervals. Make the result observable through a checksum so the compiler cannot discard an unused calculation. Check the checksum against a known or independently computed result. Do not add `volatile` to every field as a substitute for proper experimental design: that changes the memory-access semantics under study.

A small timing difference near noise is an inconclusive result. That is valid engineering evidence. Follow up with larger representative inputs and profiling, while retaining the correctness tests. Never promote an optimization whose output is wrong simply because its measured time is low.

Clamp contract and layout experiment

For `clamp_int(12,0,10)`, the result is 10. For `clamp_int(-3,0,10)`, it is 0. For invalid bounds (10,0), the function rejects the request rather than returning an arbitrary result. These tests cover the contract independently of the concept's compile-time acceptance.

The layout experiment initializes n entities with x = 0 and vx = 1. Each of r updates adds vx to x. Both implementations must finish with checksum n*r. Start with a small count that can be checked by hand, then scale the same operation. Record times only after the result passes.

A faster result for one x-only workload does not show that the layout is better for collision queries, serialization or every target CPU.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_12_4.cpp`. Pseudocode excerpts are not standalone C++ source.

```
template <std::integral T>
T clamp_int(T value, T low, T high) {
    if (high < low)
        throw std::invalid_argument("reversed bounds");
    return value < low ? low : (high < value ? high : value);
}
```

Common failure modes

A concept does not validate runtime bounds. Two layouts doing different work are not a controlled comparison. An optimized-away loop produces a timing, not useful evidence. An inconclusive measurement is better than a fabricated performance rule.

Practical assignment

Original requirement: Write a constrained generic clamp or interpolation function and benchmark two simple entity layouts. Pass: test contracts, record compiler/build mode and explain why the result is workload-specific.

1. Implement a constrained integer clamp and test its accepted types and bound policy.
2. Implement equivalent array-of-structures and structure-of-arrays updates.
3. Collect repeated Release measurements with a verified checksum and saved CSV.
4. Explain the result using only what the workload and measurement support; list one workload that could reverse it.

Required cases: below, within and above bounds; equal lower and upper bounds; reversed bounds rejected; same checksum for both layouts; zero entities handled; multiple sizes and repeated samples recorded.

Evidence to submit: Constrained function, contract tests, benchmark manifest, raw CSV and a bounded interpretation of the measurements.

Independent transfer: The hot loop changes from x-only integration to accessing every field of a single selected entity. Predict why the layout conclusion might change, then design a fair test.

Assigned primary reading: R11: Templates and Generic Programming; R15: memory hierarchy motivation.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What does a concept constrain?
2. Is structure-of-arrays universally superior?
3. Name a cost of templates beyond runtime.
4. Does satisfying a concept prove runtime bounds are valid?
5. Why must both layout variants produce an observable checked result?

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.